

- Basic Concepts
- Basic Data Types
- Operators
- Collections
- Type Conversion
- Dynamic Typing



Basic Concepts

Python Statement Terminates with End-of-Line

You can put multiple statements on the same line (separated by ;)

```
# two statements in a single  
l = 10; b = 5
```

You can also split a statement across multiple lines:

Use a backslash (\) to break a statement into two lines

```
x = 1 + 2 + 3 + 4 + \  
    5 + 6 + 7 + 8
```

Parentheses () allow expressions to span several lines naturally

```
x = (1 + 2 + 3 + 4 +  
    5 + 6 + 7 + 8)
```

- Human-readable notes or explanations added to the code
- Ignored during compilation or interpretation (no effect on execution)
- In Python, comments begin with #; everything after it is ignored by the interpreter

Single-line comment

```
01. | # This is a single line comment
02. | x = 20
```

Inline comment

```
01. | x = 20 # store 20 in x
```

Multi-line comment

No separate ways like `/* */` in C

```
01. | # This is a
02. | # multi-line
03. | # comment
04. | x = 20
```

- Keywords are the special reserved words in Python: e.g., import, True, False, def, return ...
- Their meanings are predefined by the Python interpreter
- Each keyword has a specific purpose and behavior
- They define the basic syntax and structure of Python code
- Cannot be used as names of variables, functions, classes, etc

Keywords

- The set of keywords may change across different versions of Python

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

Get Keyword List

- You can check which keywords are available in your version of Python

```
python3
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> import keyword
>>> print(keyword.kwlist)
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
>>> help('keywords')

Here is a list of the Python keywords. Enter any keyword to get more help.

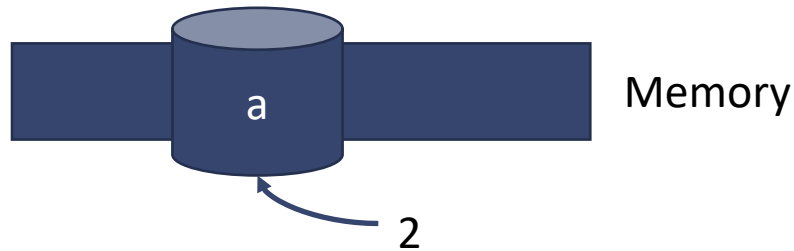
False      class      from       or
None       continue  global     pass
True       def        if         raise
and        del        import     return
as         elif       in         try
assert     else       is         while
async      except     lambda    with
await      finally   nonlocal  yield
break      for       not

>>> █
```

Variables

- A **variable** is a named storage location in memory that holds a value
- The value stored in a variable can change during program execution
- Think of it like a **labeled box**: you can put something in, take it out, or replace it with something new

Simple understanding: a container that holds data



Declaring Variable and Assigning Values

- Syntax: **variable_name = value**
- No need to specifying type in advance, created when you assign a value

```
01. int main() {  
02.     int a = 1;  
03.     return 0;  
04. }
```

C code

v.s.

```
01. a = 1  
02. print(type(a)) # <class 'int'>
```

Python code

The type of a variable is updated automatically when a new value is assigned

```
01. import sys  
02. a = 1  
03. print(a, type(a), sys.getsizeof(a)) # 1 <class 'int'> 28  
04. a = 'dog'  
05. print(a, type(a), sys.getsizeof(a)) # c <class 'str'> 50
```

Data Types

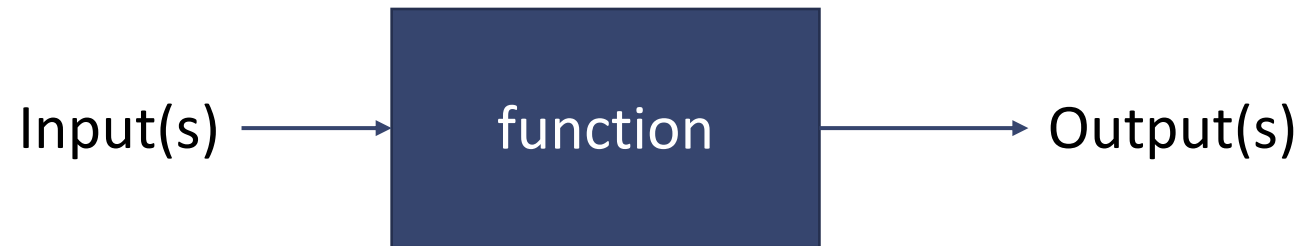
- Every value in Python is associated with a data type, which tells the interpreter how the data should be treated
- The data type determines what operations can be performed and what data structures can store the value

Common Data Types

Numbers	Boolean
String	NoneType
List	Tuple
Range	Dictionary
Set	

■ Functions

- A **function** is a reusable block of code with a specific name that performs a defined task.
- It can take inputs (called **arguments**) and return outputs (called **return values**).



■ Type of Functions

In Python, functions fall into two main categories:

- **Built-in Functions:** Already provided by Python, ready to use (e.g., `print()`, `len()`, `type()`).
- **User-Defined Functions:** Created by programmers to perform custom tasks.

Built-in Functions			
A <code>abs()</code> <code>aiter()</code> <code>all()</code> <code>any()</code> <code>anext()</code> <code>ascii()</code>	E <code>enumerate()</code> <code>eval()</code> <code>exec()</code>	L <code>len()</code> <code>list()</code> <code>locals()</code>	R <code>range()</code> <code>repr()</code> <code>reversed()</code> <code>round()</code>
B <code>bin()</code> <code>bool()</code> <code>breakpoint()</code> <code>bytearray()</code> <code>bytes()</code>	F <code>filter()</code> <code>float()</code> <code>format()</code> <code>frozenset()</code>	M <code>map()</code> <code>max()</code> <code>memoryview()</code> <code>min()</code>	S <code>set()</code> <code>setattr()</code> <code>slice()</code> <code>sorted()</code> <code>staticmethod()</code> <code>str()</code> <code>sum()</code> <code>super()</code>
C <code>callable()</code> <code>chr()</code> <code>classmethod()</code> <code>compile()</code> <code>complex()</code>	G <code>getattr()</code> <code>globals()</code>	N <code>next()</code>	T <code>tuple()</code> <code>type()</code>
D <code>delattr()</code> <code>dict()</code> <code>dir()</code> <code>divmod()</code>	H <code>hasattr()</code> <code>hash()</code> <code>help()</code> <code>hex()</code>	O <code>object()</code> <code>oct()</code> <code>open()</code> <code>ord()</code>	V <code>vars()</code>
	I <code>id()</code> <code>input()</code> <code>int()</code> <code>isinstance()</code> <code>issubclass()</code> <code>iter()</code>	P <code>pow()</code> <code>print()</code> <code>property()</code>	Z <code>zip()</code>
			<code>__import__()</code>

Input with input()

The built-in function `input()` is used to read user input

`input(prompt)`

- **Prompt:** show a message before waiting for input
- **Return Type:** Always returns a string
- **Type Conversion:** Use typecasting (e.g., `float()`) to get other data types

```
01. number = input('Enter a number: ')
02. # Enter a number: 2
03. print(number, type(number))
04. # 2 <class 'str'>
05. number_int = int(number)
06. print(number_int, type(number_int))
07. # 2 <class 'int'>
08. number_float = float(number)
09. print(number_float, type(number_float))
10. # 2.0 <class 'float'>
```



Basic Data Types

■ ■ ■ Why are Basic Data Types Important?

- They are the fundamental **building blocks** for all data in Python.
- Every piece of information, from a simple count to a complex data structure, is ultimately composed of these core types.
- Mastering them is the first step to performing calculations, making logical decisions, and representing information in your programs.

Three numeric types in Python:

- **int**: integer values (e.g., 5)
- **float**: floating point values (e.g., 3.14)
- **complex**: complex numbers (e.g., 1+2j), real part + imaginary part

Examples:

```
1  type(5)           # int
2  type(5.0)         # float
3  type(1+2j)        # complex
```

Use bool data type to represent boolean values (True, False).

Often the result of comparison operators.

Widely used in conditional statements and loops.

Examples:

```
1 print(5 > 3)           # True
2 print(bool(0))         # False
3 print(bool("hi"))      # True
```

Practice: Basic Data Types

Predict the output of the following lines of code. What is the data type of each result?

```
1 # Prediction 1
2 a = 15
3 print(type(a))
4
5 # Prediction 2
6 b = a / 2
7 print(type(b))
8
9 # Prediction 3
0 c = " "
1 print(bool(c))
2
3 # Prediction 4
4 d = 0.0
5 print(bool(d))
```



Operators

Why are Operators Important?

- Operators are the "verbs" of programming; they let you **manipulate and act on your data**.
- Without them, you could store information but couldn't perform calculations, make comparisons, or combine logical conditions.
- They are the essential tools for building expressions and implementing algorithms.

- Special symbols to perform specific operations on operands, then return a result
 - unary: operates on one operand: $-a$
 - binary: operates on two operands: $a + b$
- Category:
 - Arithmetic Operators
 - Relational Operators
 - Logical Operators
 - Bitwise Operators
 - Assignment Operators
 - Membership Operators
 - Identity Operators

Arithmetic Operators

- 7 basic binary arithmetic operators, 2 unary operators.

Operator	Name	Description
$a + b$	Addition	Adds two operands
$a - b$	Subtraction	Subtracts second operand from first
$a * b$	Multiplication	Multiplies two operands
a / b	Division (float)	Divides first operand by second, returns float
$a // b$	Floor division	Divides and returns the floor of the quotient
$a \% b$	Modulus	Returns the remainder after division
$a ** b$	Exponentiation	Raises first operand to the power of second
$-a$	Unary minus	Negate the operand
$+a$	Unary plus	Returns the same number (rarely used)

Relational Operators

Characteristics:

- used to compare the values of two operands
- output a boolean value (True or False)

Operator	Description	Operator	Description
$a == b$	Equal	$a != b$	Not equal
$a < b$	Less than	$a > b$	Greater than
$a \leq b$	Less or equal	$a \geq b$	Greater or equal

Examples

```
01. 25 < 1 # False
02. 2 == 2 # True
03. 3 != 2 # True
```

String-together multiple comparisons

```
01. 15 < 25 < 30 # True
02. 15 > 1 < 2 # True
03. 15 > 1 < 0 # False
```

Logical Operators

- Used to combine boolean values with "and", "or", and "not"

Operator	Description
and	Returns <code>True</code> only if both operands are <code>True</code> , otherwise <code>False</code>
or	Returns <code>True</code> if at least one operand is <code>True</code> , otherwise <code>False</code>
not	Returns <code>True</code> if operand is <code>False</code> , and <code>False</code> if operand is <code>True</code>

Examples:

```
1 x = 4
2 (x < 6) and (x > 2) # True
3 (x > 10) or (x % 2 == 0) # True
4 not (x < 5) # False
```

Bitwise Operators

- 6 bitwise operators: operates on binary representation

Operator	Name	Description
$a \& b$	Bitwise AND	Returns 1 if both bits are 1
$a b$	Bitwise OR	Returns 1 if at least one bit is 1
$a \wedge b$	Bitwise XOR	Returns 1 if bits are different
$a \ll n$	Left shift	Shifts bits to left by n positions
$a \gg n$	Right shift	Shifts bits to right by n positions
$\sim a$	Bitwise NOT	Inverts all bits

Bitwise Operators: Binary Representation

- Binary: a base-2 number system that uses two states 0 and 1 to represent a number

2^3	2^2	2^1	2^0
1	0	1	0

$$10 (1010) = 2^3 * 1 + 2^2 * 0 + 2^1 * 1 + 2^0 * 0$$

2^3	2^2	2^1	2^0
0	1	0	0

$$4 (100) = 2^2 * 1 + 2^1 * 0 + 2^0 * 0$$

Bitwise Operators: In-Depth Understanding

Understanding bitwise operations:

- Represent the numbers in binary
- Perform logical operation on each bit

Example: bitwise OR on 6 and 8 is 14

```
01. print(6 | 8)
```

Observe the binary representation with bin()

```
01. print(bin(6))      # 0b110
02. print(bin(8))      # 0b1000
03. print(bin(14))     # 0b1110
```

0	1	1	0
1	0	0	0
1	1	1	0

Practice: Bitwise Operators (Binary)

- $0001 \& 1111 = 0001$
- $1101 \& 0010 = 0000$
- $0010 | 1110 = 1110$
- $0100 | 1001 = 1101$
- $0010 \wedge 1101 = 1111$
- $1100 \wedge 1110 = 0010$
- $1100 \ll 1 = 11000$
- $1101 \gg 1 = 110$

Operator	Name	Description
$a \& b$	Bitwise AND	Returns 1 if both bits are 1
$a b$	Bitwise OR	Returns 1 if at least one bit is 1
$a \wedge b$	Bitwise XOR	Returns 1 if bits are different
$a \ll n$	Left shift	Shifts bits to left by n positions
$a \gg n$	Right shift	Shifts bits to right by n positions
$\sim a$	Bitwise NOT	Inverts all bits

Practice: Bitwise Operators (Decimal)

- $7 \& 5$ ➤ $111 \& 101 = 101 = 5$
- $20 | 5$ ➤ $10100 | 101 = 10101 = 21$
- $20 \wedge 5$ ➤ $10100 \wedge 101 = 10001 = 17$
- $20 >> 3$ ➤ $10100 >> 3 = 10 = 2$
- $7 >> 1$ ➤ $111 >> 1 = 11 = 3$
- $20 << 2$ ➤ $10100 << 2 = 1010000 = 80$

Operator	Name	Description
$a \& b$	Bitwise AND	Returns 1 if both bits are 1
$a b$	Bitwise OR	Returns 1 if at least one bit is 1
$a \wedge b$	Bitwise XOR	Returns 1 if bits are different
$a << n$	Left shift	Shifts bits to left by n positions
$a >> n$	Right shift	Shifts bits to right by n positions
$\sim a$	Bitwise NOT	Inverts all bits

Assignment Operators

Shortcuts for combining operation and assignment

Operator	Description	Operator	Description
$a+ = b$	$a = a + b$	$a- = b$	$a = a - b$
$a* = b$	$a = a * b$	$a// = b$	$a = a // b$
$a\% = b$	$a = a \% b$	$a* * = b$	$a = a * * b$

$a (\cdot) = b$ is equivalent to $a = a (\cdot) b$

```
01. a = 3
02. a += 2
03. print(a) # 5
```

```
01. a = 3
02. a *= 4
03. print(a) # 12
```

Practice: Assignment Operators

What is the output of the following code?

```
a = 1
a += 2
a * 4
a - 1
a /= 2
print(a)
```

✓ 0.0s

Identity Operators

Operator	Description
<code>is</code>	Test if two variables refer to the same object
<code>is not</code>	Test if two variables refer to different objects

Examples:

```
1 a = [1, 2]
2 b = a
3 print(a is b)    # True
4 print(a == b)    # True
```

Membership Operators

Test whether a value or variable is in a collection (e.g., string, list, tuple, set and dictionary)

Operator	Description
<code>in</code>	Test if a value is in a sequence
<code>not in</code>	Test if a value is not in a sequence

```
01. 1 in [1, 2, 3]      # True
02. 1 in (1, 2, 3)     # True
03. 3 not in [1, 2, 3] # False
04. 'a' in 'abc'       # True
```

Operators Precedence

Combined use of operators:

- $1 + 2 * 3 - 5 \% 2 = ?$
- $1 + 2 \& 5 * 2 = ?$

What is the order of calculation?

- **From higher precedence to lower precedence**
- **From left to right**

Some patterns

- Arithmetic > Bitwise > Comparison > Logical
- Unary > Binary

Operators	Meaning
<code>()</code>	Parentheses
<code>**</code>	Exponent
<code>+x</code> , <code>-x</code> , <code>~x</code>	Unary plus, Unary minus, Bitwise NOT
<code>*</code> , <code>/</code> , <code>//</code> , <code>%</code>	Multiplication, Division, Floor division, Modulus
<code>+</code> , <code>-</code>	Addition, Subtraction
<code><<</code> , <code>>></code>	Bitwise shift operators
<code>&</code>	Bitwise AND
<code>^</code>	Bitwise XOR
<code> </code>	Bitwise OR
<code>==</code> , <code>!=</code> , <code>></code> , <code>>=</code> , <code><</code> , <code><=</code> , <code>is</code> , <code>is not</code> , <code>in</code> , <code>not in</code>	Comparisons, Identity, Membership operators
<code>not</code>	Logical NOT
<code>and</code>	Logical AND
<code>or</code>	Logical OR

Practice: Operators

What is the final value of result? Trace the code step-by-step, paying close attention to operator precedence and assignment

```
1  x = 10
2  y = 4
3  z = 2
4  # Step 1
5  x += y * z
6  # x is now?
7  # Step 2
8  y = x % 9
9  # y is now?
10 # Step 3
11 result = (x > 15) and (y == 0)
12 # What is result?
13 print(result)
```



Collections

■ ■ ■ Why are Collections Important?

- Basic data types can only hold one value at a time. Collections (or data structures) are essential for **storing and organizing groups of related data**.
- They allow you to manage complex information, such as a list of students, a set of unique items, or a mapping of words to their definitions.
- Choosing the right collection for the job is a key programming skill.

- Two types of objects in Python: **Mutable** and **Immutable**
- A mutable object can be changed after it is created
 - list, dict, set
- An Immutable object cannot be changed after it is created
 - int, float, bool, str, tuple
- Mutable objects are not hashable (i.e., cannot be keys in dict, elements in set...)

Characteristics

- An ordered collection of items (elements)
- Separated by commas, enclosed within square brackets ([])
- The values can be of any data type
- Lists are **mutable**.

```
01. a = [1, 2, 5, 4]
02. print(type(a)) # <class 'list'>
```

```
01. # empty list
02. my_list = []
03.
04. # list of integers
05. my_list = [1, 2, 3]
06.
07. # list with mixed datatypes
08. my_list = [1, "Hello", [], 1.2]
```

Tuple

Characteristics

- An ordered collection of items
- Separated by commas, enclosed within parentheses ()
- The values can be of any data type
- Tuples are **immutable**

```
01. a = (1, 2, 5, 4)
02. print(type(a)) # <class 'tuple'>
```

Notice: **what actually matters is comma (,) , not parentheses**

```
01. a = 1, 2, 3
02. print(a, type(a)) # (1, 2, 3) <class 'tuple'>
```

Can have different typed items

```
01. a = (1, 2, 5, 'dog', [], 6.7)
```

Characteristics

- A contiguous series of characters enclosed in quotes (single, double, or triple).
- Triple quotes are used to define multi-line strings.
- Strings are **immutable**, so they cannot be changed after creation

Examples:

```
1 s1 = "Hello, World!"
2 s2 = """This is a
3 multi-line string."""
4 print(type(s1)) # <class 'str'>
```

String Formatting

Two common ways to format strings by embedding variables:

- **format()** method: Uses curly braces {} as placeholders for variables that are passed into the method.
- **String Formatting Operator %** : Similar to C's printf(), that uses placeholders like %s (string) and %d (integer).

Examples:

```
1 language = "Python"
2 year = 1991
3 # Using the .format() method
4 print("{} , was developed in {}".format(language , year))
5 # Using the % operator
6 print("%s was developed in %d." % (language , year))
```

Characteristics:

- An **unordered** collection of items
- Every element is unique (no duplicates).
- Separated by commas, enclosed within curly braces {}.
- Set **items must be of an immutable (hashable) type**.

Examples:

```
1  # A set of integers
2  my_set = {1, 2, 5, 4}
3  print(type(my_set))  # <class 'set'>
4  # Create an empty set with set()
5  empty_set = set()
6  # Note: {} creates an empty dictionary, not a set
```

Sets are Mutable

Can be modified after creation by adding or removing elements.

Adding Elements:

- `add(elem)`: Adds a single element.
- `update(iterable)`: Adds all elements from an iterable (like a list or another set). Duplicates are automatically discarded.

Removing Elements:

- `remove(elem)`: Removes an element; raises a `KeyError` if the element is not found.
- `discard(elem)`: Removes an element if it is present, but does nothing if it's not.

Examples:

```
1 my_set = {1, 2, 3}
2 my_set.add(4) # {1, 2, 3, 4}
3 my_set.update([2, 5, 6]) # {1, 2, 3, 4, 5, 6}
```

Hashable set? Frozen Set!

- A set that is **immutable** (thus hashable)
- `frozenset()`: an inbuilt function which takes an iterable objects as argument and make them immutable
- Usage: used as a dictionary key
- Set operations are supported (except for removing and adding methods)

```
>>> a = {1, 2, 3}
>>> b = frozenset(a)
>>> b
frozenset({1, 2, 3})
>>> c = {b}
>>> c
{frozenset({1, 2, 3})}
```

Characteristics:

- A collection of key-value pairs (**mutable**).
- Keys must be unique and of an immutable (hashable) type.
- Values can be of any data type and can be duplicated.
- Enclosed by curly braces {}, with key-value pairs separated by colons :

Examples:

```
1 # A dictionary of student ages
2 person_age = {"John": 21, "Kim": 35, "Alex": 26}
3 print(type(person_age)) # <class 'dict'>
4 # Creating an empty dictionary
5 empty_dict = {}
```

■ Accessing and Modifying Dictionaries

Accessing Values:

- Use square brackets ['key'] to access the value. This will raise a KeyError if the key does not exist.
- Use the get('key') method, which returns None (or a default value) if the key is not found.

Adding or Updating:

- Assign a value to a **new key** to add a new pair.
- Assign a **new value** to an existing key to update it.

Examples:

```
1 person_age = {"Kim": 35}
2 print(person_age['Kim'])          # 35
3 print(person_age.get('Alex'))    # None
4 person_age['Steve'] = 32 # Adding a new element
5 # Updating an existing element
6 person_age['Kim'] = 30
7 print(person_age) # {'Kim': 30, 'Steve': 32}
```

General Collection Methods

Characteristics:

- Apply to general collections, such as lists, tuples, and strings.
- Return a new value and do not modify the original collection.
- More readable and concise than writing a loop

Method	Description
<code>all()</code>	Judge if all element is True
<code>any()</code>	Judge if any element is True
<code>len()</code>	Return the length of a collection
<code>max()</code>	Return the maximum of a collection
<code>min()</code>	Return the minimum of a collection
<code>sum()</code>	Return the sum of a collection
<code>sorted()</code>	Return a sorted list of a collection



Type Conversion

■ Why is Type Conversion Important?

- Data doesn't always come in the format you need. Type conversion is an essential skill for **manipulating and preparing data**.
- It's crucial for tasks like converting user input from 'input()' (which is always a string) into a number for calculations.
- It also allows you to leverage the features of different data types, like converting a list into a set to easily find unique elements.

Type Conversion

The process of converting an object from one data type to another.

Implicit Conversion (Coercion):

- Python automatically converts one data type to another to avoid data loss.
- For example, adding an int to a float will result in a float.

Explicit Conversion (Casting):

- Manually converted by the programmer using built-in functions like `int()`, `float()`, `str()`, `list()`, etc

Examples:

```
1  # Implicit conversion
2  c = 2 + 4.5; print(type(c)) # <class 'float'>
3  # Explicit conversion
4  a = int(3.9); print(a)      # <class 'int'>
5  s = str(123); print(type(s)) # <class 'str'>
```

Common Explicit Type Conversions

Function	Description	Example
<code>int()</code>	Convert to integer	<code>int('123')</code> to <code>123</code>
<code>float()</code>	Convert to float	<code>float('123.4')</code> to <code>123.4</code>
<code>tuple()</code>	Convert to tuple	<code>tuple([1,2,3])</code> to <code>(1,2,3)</code>
<code>set()</code>	Convert to set (remove duplicates)	<code>set([1,2,2,3])</code> to <code>1,2,3</code>
<code>dict()</code>	Create dict from key-value pairs	<code>dict([('a',1),('b',2)])</code> to <code>'a':1, 'b':2</code>
<code>complex()</code>	Create complex number	<code>complex(2,3)</code> to <code>(2+3j)</code>
<code>bool()</code>	Convert to boolean	<code>bool(0)</code> to <code>False</code>
<code>bytes()</code>	Convert to bytes	<code>bytes('hello', 'utf-8')</code> to <code>b'hello'</code>
<code>frozenset()</code>	Create immutable set	<code>frozenset([1,2,3])</code> to <code>frozenset(1)</code>

Special Notes:

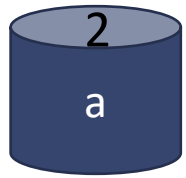
```
1 # Dict conversion requires proper key-value pairs
2 dict(['key', 'value'], ('a', 1)) # Valid
3 dict(['ab', 'cd']) # ValueError: dictionary update
    sequence element #0 has length 2; 2 is not 2
```



Dynamic Typing

Questions on Variables

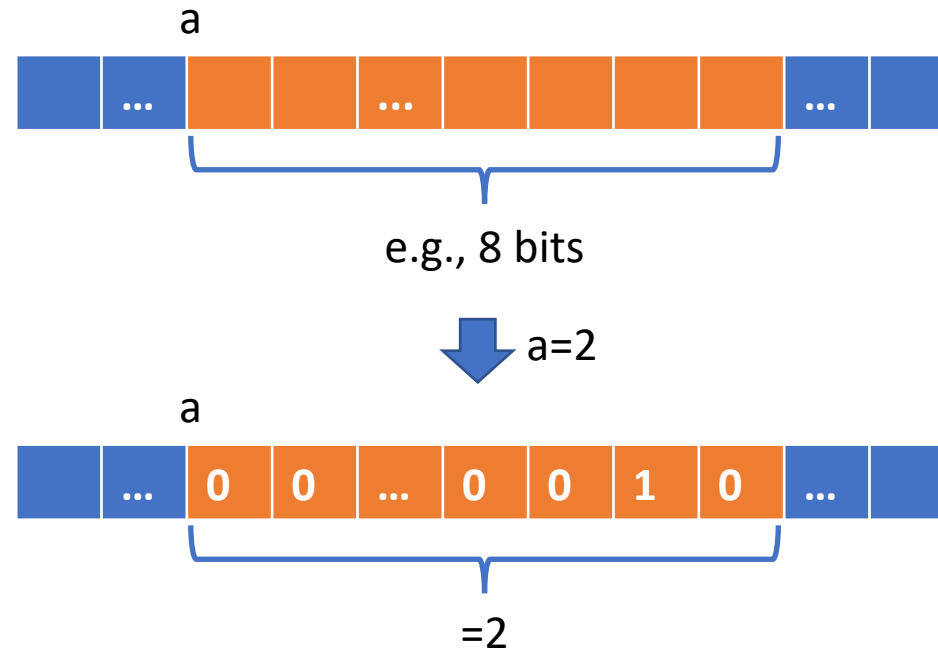
- Question 1: How does a variable hold data?
- Question 2: What is happening during value assignment?



Fundamental Idea

Question 1: How does a variable hold data?

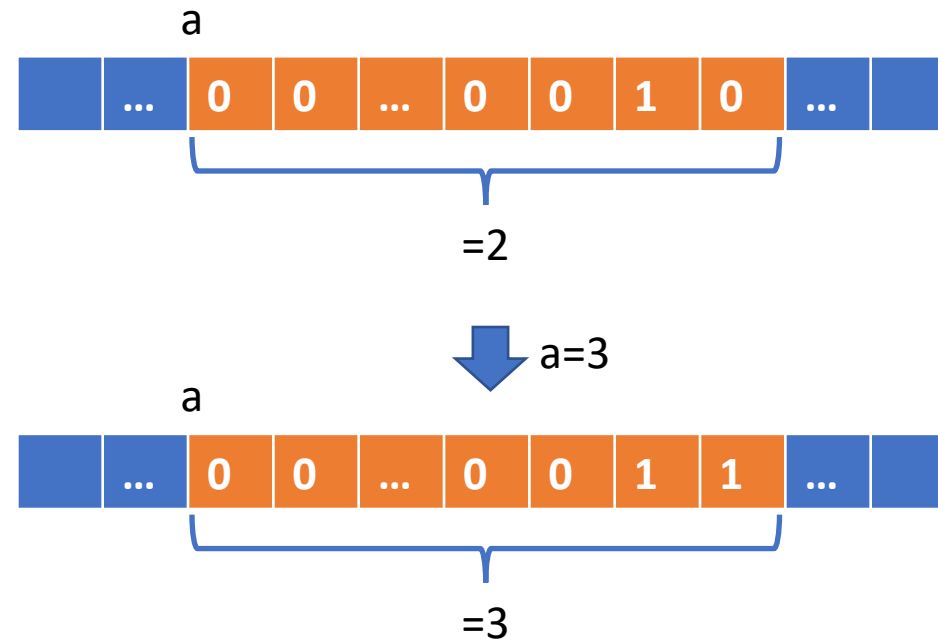
- The computer allocates memory space to the variable
- The assigned value is written into that memory location



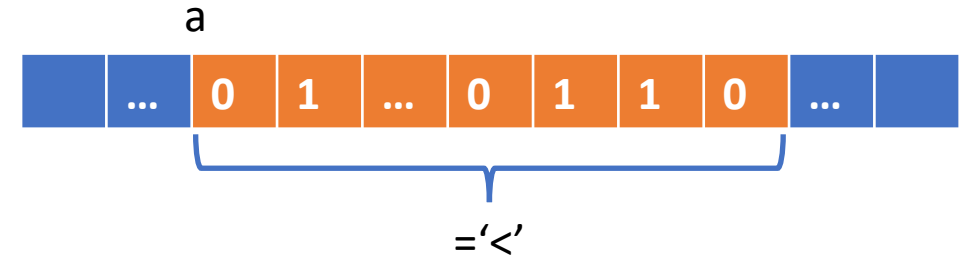
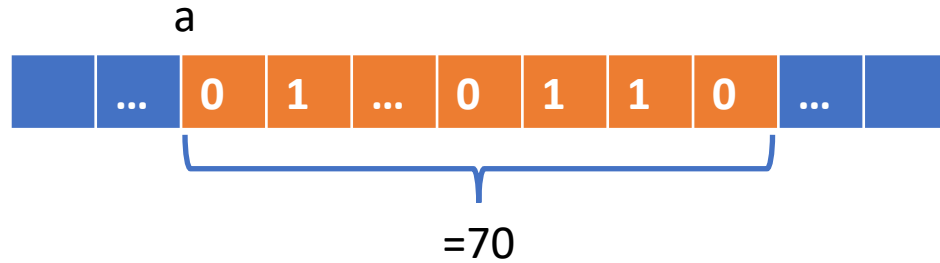
Fundamental Idea

Question 2: What is happening during value assignment?

- The computer clears the previous value stored in memory
- The new value is then written into that memory location



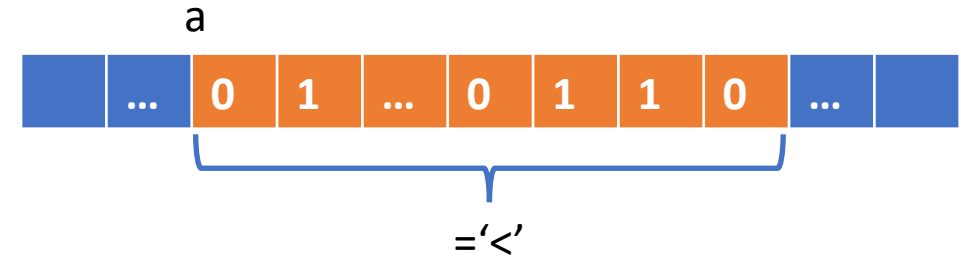
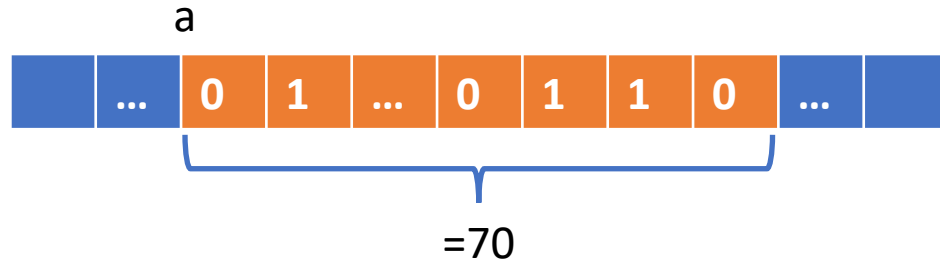
Question 3: How is type information stored?



Same binary data can represent different types

Type Information

Question 3: How is type information stored?



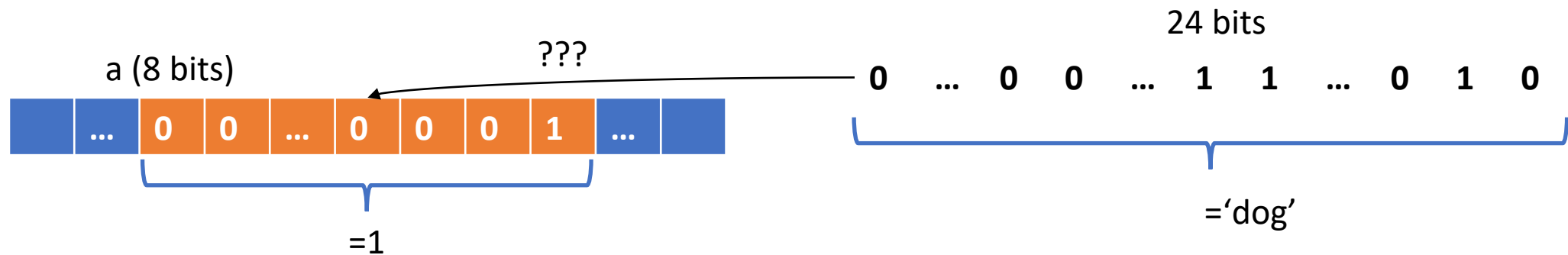
Type information is **associated with the variable**

Dynamic Typing in Python

Question 4: Where is type information in Python?

Question 5: How are values with different size stored?

```
01. import sys
02. a = 1
03. print(a, type(a), sys.getsizeof(a)) # 1 <class 'int'> 28
04. a = 'dog'
05. print(a, type(a), sys.getsizeof(a)) # c <class 'str'> 50
```

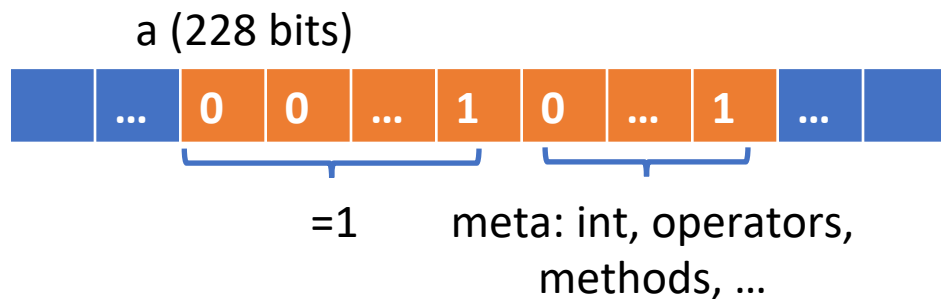


Dynamic Typing in Python

Question 4: Where is type information in Python?

- Type information is attached to each object, not to the variable name

```
01. import sys
02. a = 1
03. print(a, type(a), sys.getsizeof(a)) # 1 <class 'int'> 28
04. a = 'dog'
05. print(a, type(a), sys.getsizeof(a)) # c <class 'str'> 50
```

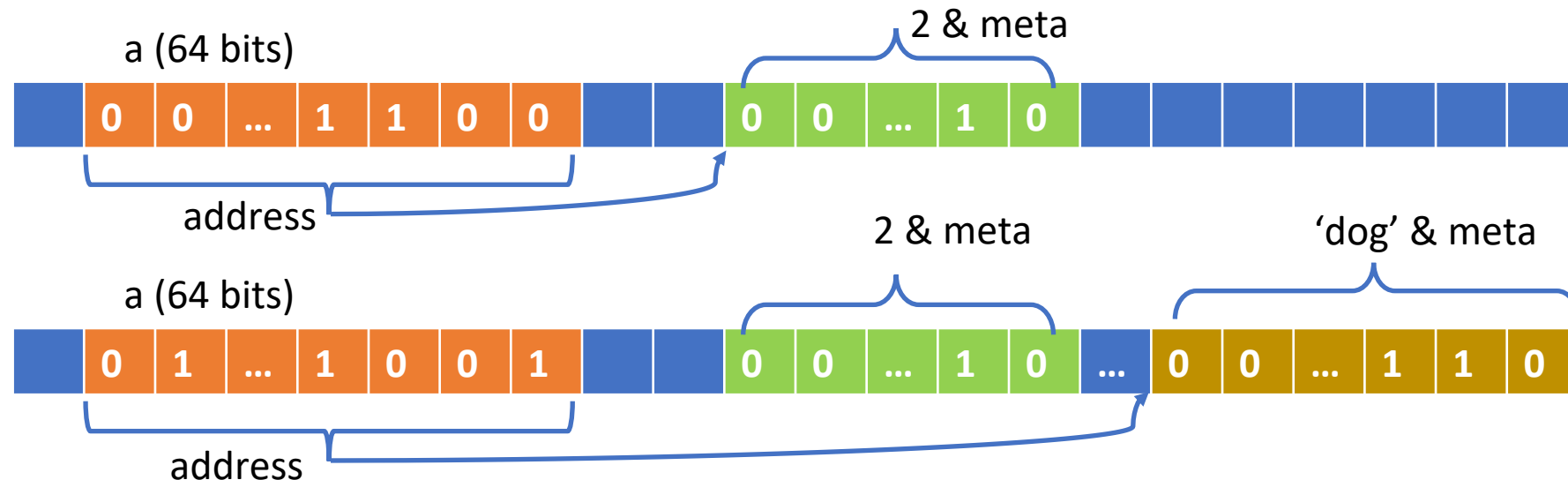


Brings extra memory cost
8 bytes in C but 28 bytes in Python

Dynamic Typing in Python

Question 5: How are values with different sizes stored?

- The actual value is stored somewhere else in memory
- A variable only keeps **a reference (an address)** to that value
- Assigning a new value updates the reference to point to a different address



- `id()` function: : returns the unique identity of an object (in CPython, this corresponds to the object's memory address)

Question 6: What happens when one variable is assigned to another?

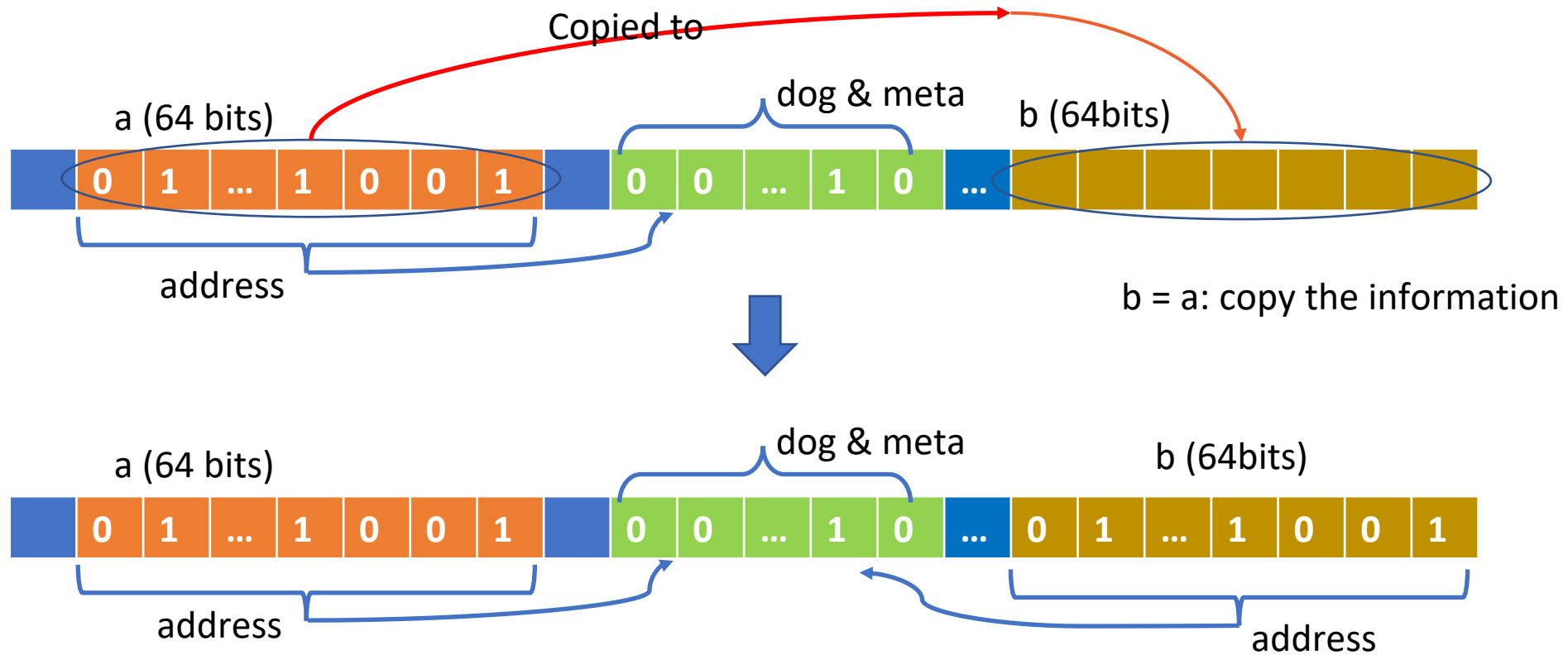
Create a new object? No.

```
>>> a = ['a', 'b', 'c']  
>>> b = a  
>>> print(b)  
['a', 'b', 'c']
```

Dynamic Typing in Python

Question 6: What happens when one variable is assigned to another?

- Both variables reference the same memory address



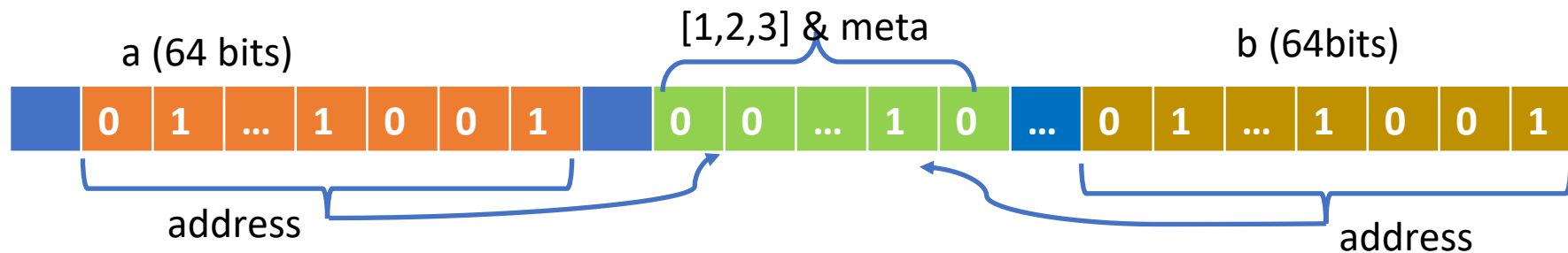
Assignment of Mutable Objects

Assigning an existing object to a new variable does not create a copy

It only creates an **alias**

```
01. a = [1, 2, 3]
02. b = a # aliasing
03. b[0] = 0 # update through b
04. print(a) # [0, 2, 3]
05. print(id(a), id(b)) # 4412362176 4412362176
```

id is the same — modification on b will reflect on a



Assignment of Mutable Objects

Cloning: creating an exact copy that is independent of the original

- `copy()` function

```
01. a = [1, 2, 3]
02. b = a.copy()
03. print(a, b) # [1, 2, 3] [1, 2, 3]
04. b[0] = 10
05. print(a, b) # [1, 2, 3] [10, 2, 3]
```

- Using Slicing also create a copy

```
01. a = [1, 2, 3]
02. b = a[:] # cloning
03. print(a, b) # [1, 2, 3] [1, 2, 3]
04. b[0] = 10
05. print(a, b) # [1, 2, 3] [10, 2, 3]
```

Identity Operators

Check two values (or variables) refer to the same object in memory

Operator	Description
a is b	True if a and b are identical objects
a is not b	True if a and b are not identical objects

```
01. a = [1, 2, 3]
02. b = [1, 2, 3]
03. a == b # True
04. a is b # False
```

```
01. a = [1, 2, 3]
02. b = a
03. a == b # True
04. a is b # True
```

Equality (==) does not imply identity (is)

What is the output? Why?

```
1  lst = [[]] * 5
2  lst[0].append(1)
3  print(lst)
```

Practice: Compare Data Representation

You need to store one student record with three fields:

```
student_id = "2026001", name = "Alice", score = 95
```

Consider the following three representations:

```
record_list = ["2026001", "Alice", 95]
```

```
record_tuple = ("2026001", "Alice", 95)
```

```
record_dict = {"student_id": "2026001", "name": "Alice", "score": 95}
```

Your tasks:

1. Write a function `total_size(obj)` to estimate the size of each record, including the container and its direct elements. (Hint: you may use `sys.getsizeof()`)
2. Compare 3 representations in terms of memory usage, readability, mutability
3. Question: Suppose you need to store 10,000 student records. Which representation would you choose, and why?
4. Can you propose a better representation that uses less memory while maintaining good readability and mutability? (Hint: You may not use NumPy or Pandas. Consider Python built-in or standard-library features.)

Revisiting Today's Content

- **Basic Concepts**
- **Basic Data Types:** The fundamental building blocks (integers, floats, booleans).
- **Operators:** The “verbs” that allow us to manipulate and compare data.
- **Collections:** Essential for organizing and storing groups of data (lists, tuples, strings, sets, dictionaries).
- **Type Conversion:** Crucial for adapting data to different needs and leveraging various data type features.
- **Dynamic Typing:** determine variable types at runtime, allowing flexible programming but incurring extra overhead and requiring attention to type-related errors

Why do we need them?

- These concepts form the bedrock of Python programming, enabling you to build increasingly complex and powerful applications

Python Programming with Artificial Intelligence

Thank you for listening!

